

Inkrementelles Restklassen-Verfahren zur Erzeugung von Primzahlen

Ralf Sieg

2026

Abstract

Dieses Paper stellt ein inkrementelles Restklassenverfahren zur Erzeugung von Primzahlen vor, das ohne Divisionen, Multiplikationen oder explizite Modulo-Operationen auskommt. Stattdessen werden die Restwerte modulo bereits bekannter Primzahlen schrittweise fortgeführt. Das Verfahren ist einfach, zustandsarm und unbegrenzt fortsetzbar, da es nur die minimal notwendige Information für die inkrementelle Fortschreibung speichert. Obwohl es asymptotisch langsamer ist als klassische Siebverfahren, bietet es konzeptionelle Einfachheit und praktische Vorteile in Streaming- und Embedded-Umgebungen, in denen inkrementelle Berechnung mit geringem Speicherbedarf entscheidend ist. Eine vollständige Komplexitätsanalyse wird bereitgestellt.

1 Einleitung

Das hier vorgestellte Verfahren erzeugt Primzahlen durch das sequentielle Durchlaufen aller natürlichen Zahlen ab 2, ohne dabei Divisionen, Multiplikationen oder Modulo-Berechnungen explizit auszuführen. Stattdessen werden die Restwerte der Division durch bereits bekannte Primzahlen inkrementell fortgeführt. Dadurch entsteht ein einfaches, endlos fortsetzbares Verfahren zur Primzahlbestimmung.

Ziffern werden zur besseren Lesbarkeit in Hochkommata gesetzt („0“, „1“, ...), während Zahlen ohne Hochkommata geschrieben werden (2, 3, 4 ...).

2 Grundidee: Restklassen-Vektor über Primzahlen

Das Verfahren verwendet kein Stellenwertsystem wie das Dezimalsystem, sondern einen Restklassen-Vektor, im Folgenden *Primzahlen-System* genannt. Jede Stelle entspricht einer Primzahl p und speichert den Restwert der aktuellen Zahl $n \bmod p$.

Für jede Primzahl p existiert eine Stelle mit einem Ziffernvorrat von „0“ bis „ $p-1$ “.

Damit repräsentiert jede Stelle den Wert $n \bmod p$.

3 Inkrementieren im Primzahlen-System

Beim Übergang von n zu $n + 1$ gilt:

$$(n + 1) \bmod p = (n \bmod p + 1) \bmod p.$$

Daraus folgt die zentrale Regel:

Alle Stellen des Systems werden beim Inkrementieren um 1 erhöht. Erreicht eine Stelle ihren Maximalwert, springt sie auf „0“. Neue Stellen beginnen immer bei „0“.

4 Primzahlerkennung

Eine Zahl n ist genau dann prim, wenn sie durch keine kleinere Primzahl teilbar ist. Im Primzahlen-System bedeutet das:

Wenn nach dem Inkrementieren keine Stelle den Wert „0“ enthält, ist die Zahl prim.

Für jede gefundene Primzahl wird eine neue Stelle mit Ziffer „0“ hinzugefügt. Damit sind beim Erreichen einer Zahl n alle Primzahlen $\leq n$ bereits vorhanden.

5 Darstellung im Dezimalsystem

Zur Visualisierung werden Dezimaldarstellung und Restklassen-Vektor parallel geführt.

Dezimalsystem	Primzahl der Stelle				
	2	3	5	7	11
2	0				
3	1	0			
4	0	1			
5	1	2	0		
6	0	0	1		
7	1	1	2	0	
8	0	2	3	1	
9	1	0	4	2	
10	0	1	0	3	
11	1	2	1	4	0

6 Komplexitätsanalyse

Das Verfahren führt für jede Zahl n alle bisher bekannten Primzahlen p als Stellen im Restklassen-Vektor.

6.1 Zeitkomplexität

Die Anzahl der Stellen entspricht der Anzahl der Primzahlen $\leq n$:

$$\pi(n) \sim \frac{n}{\ln n}.$$

Gesamtkosten bis N :

$$T(N) = \sum_{n=2}^N \pi(n) = O(N \cdot \pi(N)) = O\left(\frac{N^2}{\ln N}\right).$$

Interpretation

- Quadratisch geteilt durch einen Logarithmus.
- Langsamer als das Sieb des Eratosthenes ($O(N \log \log N)$).
- Extrem einfach, inkrementell, zustandsarm und endlos fortsetzbar.

6.2 Speicherkomplexität

Eine Stelle pro Primzahl:

$$S(N) = \pi(N) = O\left(\frac{N}{\ln N}\right).$$

6.3 Vergleich mit klassischen Verfahren

Verfahren	Zeit	Speicher	Bemerkung
Inkrementelles Restklassen-Verfahren	$O(N^2 / \ln N)$	$O(N / \ln N)$	Einfach, inkrementell, ohne Division
Sieb des Eratosthenes	$O(N \log \log N)$	$O(N)$	Sehr schnell, nicht inkrementell
Trial Division	$O(N\sqrt{N})$	$O(1)$	Extrem langsam
Segmentiertes Sieb	$O(N \log \log N)$	$O(N)$	Speicheroptimiert

7 Stärken und Schwächen

7.1 Stärken

- Keine Division, Multiplikation oder Modulo.
- Echt inkrementell.
- Unendlich fortsetzbar.
- Sehr einfache Implementierung.

7.2 Schwächen

- Asymptotisch langsamer als moderne Siebe.
- Für große Bereiche nicht konkurrenzfähig.

8 Fazit

Das inkrementelle Restklassen-Verfahren ist ein ungewöhnlich einfaches, vollständig inkrementelles Primzahlverfahren. Es ist nicht für maximale Geschwindigkeit optimiert, bietet aber einzigartige Vorteile für Streaming- und Embedded-Szenarien.

Lizenz

Licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). Full license text: <https://creativecommons.org/licenses/by/4.0/legalcode>